

Semantic Extractor

Embedding-Powered Semantic Search — Technical Project Report

Simone Lungarella

2026-09-24

Contents

1	Executive summary	2
2	Program need	3
2.1	Scenario	3
2.2	Objectives and success measures	3
2.3	Why AI was appropriate	3
2.4	Constraints	3
3	Solution and technology	4
3.1	Architecture	4
3.2	Import workflow	4
3.3	Search workflow	4
3.4	Model and provider choices	4
4	Implementation and work products	6
4.1	Application and API implementation	6
4.2	Provider-neutral embedding integration	6
4.3	Redis vector storage and retrieval	6
4.4	Repeatable development and demonstration workflow	6
5	Delivery approach	8
6	Evaluation and results	9
6.1	Measured environment	9
6.2	Retrieval observations	9
6.3	Interpretation	9
7	Trustworthy AI, ethics, and governance	10
7.1	Misleading relevance	10
7.2	Ambiguous or context-poor data	10
7.3	Corpus coverage and evaluation bias	10
7.4	Model and vector compatibility	10
7.5	Credential exposure	10
7.6	Data privacy and intellectual property	10
7.7	Reproducibility drift	10
8	Technical outcomes and lessons learned	11
9	Supporting work products	11
10	References	12

1 Executive summary

I designed and implemented **Semantic Extractor**, a small Java proof of concept that evaluates whether embedding-based retrieval can improve discovery in a catalog when users know the concept they need but not the catalog's exact terminology. The application compares a conventional lexical prefix search with semantic vector search over the same catalog of synthetic Linux dictionary terms.

This is a self-directed proof of concept rather than a client deployment. All implementation artifacts and measurements discussed in this report were produced for the project. No client data, personal data, internal IBM information, or credentials are included.

The delivered workflow uses Spring Boot for its REST API, IBM Granite Embedding 30M English through Ollama for local embedding inference, and Redis Stack for term storage and cosine vector search. During import, the application normalizes and filters terms, creates embeddings in batches, converts each vector to binary `FLOAT32`, stores it in a Redis hash, and creates a 384-dimensional vector index. At query time, it embeds a natural-language description with the same model and returns the nearest catalog terms with similarity scores. A separate lexical endpoint provides the non-AI baseline.

The implemented work includes the application structure, REST APIs, ingestion pipeline, embedding abstraction, Ollama and watsonx.ai integrations, binary vector encoding, Redis index management, semantic retrieval, repeatable local environment, and technical documentation. In the validated local run, 856 terms were indexed with zero Redis indexing failures. Prepared semantic queries retrieved `library`, `developer`, `hammer`, and `hospital` in the first five results even though those terms did not appear in the respective queries.

The application integrates with watsonx.ai through the official Java SDK and targets **ibm/granite-embedding-278m-multilingual** for cloud-hosted embeddings. For repeatable development and local validation, I used IBM Granite Embedding 30M English through Ollama. The measurements in this report come from that local run; they are not presented as watsonx.ai execution results.

2 Program need

2.1 Scenario

The scenario represents a catalog-search capability for users who can describe what they need but may not know its registered name. A conventional prefix search works when the user already knows that a term begins with `lib`, for example, but cannot infer that the description “a place where books are borrowed” refers to `library`.

The Linux dictionary is deliberately synthetic, non-sensitive source data. It keeps the experiment reproducible without introducing client, personal, or proprietary information.

2.2 Objectives and success measures

I defined the MVP objectives as follows:

1. provide one repeatable import workflow over a bounded catalog;
2. establish lexical prefix search as a conventional baseline;
3. generate real embeddings with an IBM model;
4. persist and search binary vectors in Redis Stack;
5. retrieve an expected term within the first five semantic results for a small set of natural-language queries; and
6. retain unsuccessful or irrelevant results as evidence of limitations rather than presenting only favourable outputs.

The main technical success measure was **expected-term recall within the first five results**. Operational checks included application and Redis health, embedding dimensions, indexed-document count, indexing failures, and import duration.

2.3 Why AI was appropriate

Lexical matching is deterministic, inexpensive, and preferable when the query contains the catalog terminology. It does not solve the selected problem when query and catalog wording differ. An embedding model is appropriate because it maps both catalog terms and natural-language queries into the same numerical space, allowing Redis to rank terms by vector proximity instead of character overlap.

The MVP intentionally does not use a text-generation model or autonomous agent. It returns retrieved catalog terms and similarity scores. This keeps the design focused on the identified retrieval problem, limits cost, and avoids generating unsupported natural-language explanations.

2.4 Constraints

The project was deliberately constrained to approximately 1,000 submitted dictionary entries, local containerized infrastructure, and a small embedding model. This reduced inference time and made the workflow suitable for a short demonstration. The corpus consists of isolated words, which have limited context and can be ambiguous. It is therefore suitable for technical validation but not for claiming production search quality.

3 Solution and technology

3.1 Architecture

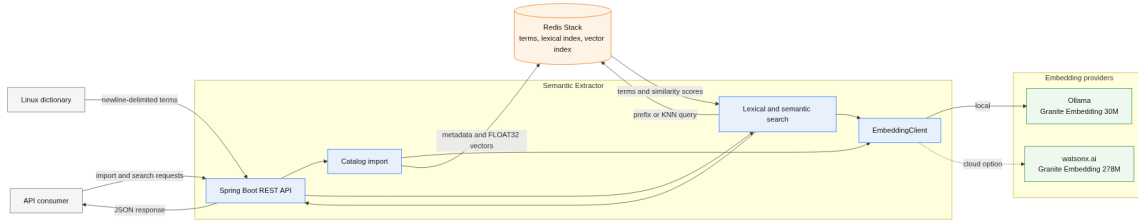


Figure 1: Semantic Extractor architecture

The public project repository contains the source code, editable Mermaid diagrams, local environment, and reproducible commands described in this report: <https://github.com/simone-lungarella/semantic-extractor>.

The application has four principal runtime components:

- a Spring Boot REST API for import, diagnostics, and search;
- an `EmbeddingClient` abstraction that isolates provider-specific inference;
- IBM Granite embeddings served through either `watsonx.ai` or local Ollama; and
- Redis Stack for hashes, lexical indexing, and vector similarity search.

Provider selection is externalized through environment configuration, while the ingestion and search services depend only on the embedding interface. This allows the same import, vector-storage, and search workflow to operate with either provider.

3.2 Import workflow

The `make import` workflow selects a deterministic sample distributed through `/usr/share/dict/words` and includes known evaluation terms that also exist in that file. The API then:

1. converts values to lowercase;
2. accepts only terms between 4 and 15 characters containing `a-z`;
3. removes duplicates while preserving order;
4. requests embeddings in batches of 100;
5. verifies vector count and provider-specific dimensions before persistence;
6. converts each Java floating-point vector to little-endian `Float32` bytes;
7. stores term metadata and its binary vector in a Redis hash; and
8. creates a Redis `FLAT` vector index using cosine distance.

I selected `FLAT` rather than an approximate HNSW index because the catalog is small. Exact search is simpler to explain and operate at this scale, while an approximate index would add tuning decisions without a meaningful MVP benefit.

3.3 Search workflow

Lexical search uses a Redis sorted set and a bounded lexicographical range to return literal prefixes without loading the complete catalog into Java.

Semantic search validates the natural-language query, generates one embedding with the same model used during import, serializes it with the same vector codec, and executes a Redis K-nearest-neighbour query. Redis returns cosine distance, which the application exposes as `1 - distance` for a more readable similarity score. The response also identifies the embedding model used.

3.4 Model and provider choices

For local validation, I chose `granite-embedding:30m`, an English IBM Granite embedding model distributed through Ollama. Its approximately 63 MB footprint and 384-dimensional output made it fast

enough for repeated local imports. The corpus and evaluation queries are English, so the larger multilingual model was not required for the MVP.

For watsonx.ai, the application uses the official `com.ibm.watsonx:watsonx-ai` Java SDK and targets `ibm/granite-embedding-278m-multilingual`. The adapter is activated by selecting the `watsonx` provider and supplying an IBM Cloud API key, a watsonx.ai project ID, and the regional service URL through environment variables. Credentials and account-specific values are not stored in source code or application defaults.

The watsonx.ai model produces 768-dimensional vectors, while the local 30M model produces 384-dimensional vectors. Dimensions are therefore part of the provider contract. Changing provider or model requires rebuilding the Redis index and regenerating the catalog embeddings; the `reset` import performs this rebuild and prevents vectors from incompatible model spaces from being mixed.

Local Ollama inference was used to make development, repeated imports, and evaluation independent of network access and cloud credentials. watsonx.ai remains an application integration rather than the source of the measurements reported in this document.

4 Implementation and work products

The following work products describe the implementation and are safe to disclose because they contain synthetic data and no secrets.

4.1 Application and API implementation

Purpose. Provide a minimal service that demonstrates ingestion, a lexical baseline, and semantic retrieval without adding unrelated UI, security, or agent-framework complexity.

Implementation. The Spring Boot controllers and services provide catalog import, lexical search, semantic search, embedding diagnostics, health reporting, request validation, and error handling.

Technical decision. I kept the APIs small and made the import accept newline-delimited `text/plain`. The application does not depend directly on the host dictionary path; the Make target reads the source and submits it. This keeps source-data acquisition outside the service while preserving a repeatable demo.

Validation. I exercised the health, import, lexical, embedding, and semantic endpoints in a running local environment. The lexical query `lib` returned `library`, while natural-language queries retrieved conceptually related terms without literal query overlap.

4.2 Provider-neutral embedding integration

Purpose. Prevent ingestion and retrieval logic from depending on one model hosting platform.

Implementation. The `EmbeddingClient` interface isolates embedding inference from import and search. One adapter invokes local Granite through the Ollama `/api/embed` endpoint. A second adapter invokes Granite embeddings through the official watsonx.ai Java SDK. A disabled provider allows lexical-only startup when no embedding provider is selected.

Technical decision. Each provider declares its model identifier and vector dimensions. Ingestion and search consume only the shared interface, keeping model-specific integration outside the catalog workflow.

Validation. The local diagnostic call returned one 384-dimensional Granite vector and exposed only five sample values rather than returning the complete embedding. The adapter also verifies that one vector is returned per input and that dimensions match configuration. The same diagnostic endpoint is available when watsonx.ai is selected, allowing credentials, model access, and the expected 768-dimensional output to be checked before catalog import.

4.3 Redis vector storage and retrieval

Purpose. Persist catalog embeddings and retrieve nearest terms efficiently without introducing another database.

Implementation. The vector layer provides little-endian `FLOAT32` encoding, binary hash-field persistence, automatic vector-index creation, dimension-aware index rebuilding, raw RediSearch KNN execution, response parsing, and similarity-score conversion.

Technical decision. I used Redis for both the lexical and vector paths so that the comparison changes the retrieval method rather than the underlying catalog or operational platform. For 384 dimensions, each vector occupies 1,536 raw bytes before database overhead.

Validation. The inspected Redis index contained 856 documents, used `FLOAT32`, dimension 384, `FLAT`, and `COSINE`, and reported zero indexing failures. Its vector-index memory was approximately 1.54 MB in the measured run.

4.4 Repeatable development and demonstration workflow

Purpose. Make the POC reproducible and understandable without requiring manual orchestration knowledge.

Implementation. The project includes a Podman Compose environment, Make targets, health checks, deterministic sample import, formatted demo output, architecture diagram, README, and a dedicated demonstration guide.

Technical decision. Ollama is an optional Compose profile with a persistent model volume. Redis also uses persistent storage. The `make demo` target runs a fixed lexical comparison and four semantic queries, while `make redis-index` shows only the vector-index fields relevant to technical evidence.

Validation. I executed the documented startup, import, index inspection, and query sequence successfully from the command line.

5 Delivery approach

I used an incremental, evidence-driven delivery approach consistent with iterative application development:

1. I constrained the use case to one retrieval problem and a synthetic catalog.
2. I implemented catalog ingestion and lexical search first, creating a non-AI baseline before adding model complexity.
3. I introduced a provider interface before integrating any hosted model, reducing coupling and preserving local and cloud options.
4. I added a diagnostic endpoint to validate inference independently from import and Redis indexing.
5. I completed the local Granite path so development and evaluation remained repeatable without external credentials.
6. I tested the system end to end and changed the original first-1,000-lines sample after observing that it contained almost exclusively early-alphabet terms and could not support meaningful evaluation.
7. I retained weak and irrelevant matches in the demonstration to avoid overstating model quality.

These iterations show how observed evidence changed the implementation. I also removed a proposed deterministic embedding stub when it became clear that it would test infrastructure but provide no meaningful semantic evidence. Using a small real Granite model produced stronger validation with little additional local cost.

The approach was pragmatic rather than tied to a named delivery framework. The relevant practices were incremental delivery, baseline comparison, separation of concerns, externalized configuration, repeatable infrastructure, and changes driven by observed results.

6 Evaluation and results

6.1 Measured environment

The validated local run used IBM Granite Embedding 30M English through Ollama and a Redis Stack FLAT cosine index. The deterministic import submitted 1,000 lines; after validation and deduplication, 856 terms were indexed. Import and embedding generation completed in approximately 13 seconds in the observed run. Redis reported zero vector-indexing failures.

The principal validated components were:

- Java 21 runtime with a Java 17 compilation target;
- Spring Boot 4.1.1;
- Ollama 0.34.4;
- Redis 7.4.7; and
- `granite-embedding:30m`, producing 384-dimensional vectors.

These versions describe the measured environment rather than minimum supported versions. Model and infrastructure updates may change latency or result ranking.

6.2 Retrieval observations

- **Library:** The query “a place where books are borrowed” returned `library` at rank 1 with a similarity score of 0.6754.
- **Developer:** The query “a person who creates computer software” returned `developer` at rank 1 with a similarity score of 0.7476.
- **Hammer:** The query “a tool used to hit a nail” returned `hammer` at rank 3 with a similarity score of 0.6245.
- **Hospital:** The query “a place where sick people receive treatment” returned `hospital` at rank 1 with a similarity score of 0.7151.

The expected term appeared in the first five results for all four prepared queries. Three appeared first and one appeared third. This is a measured result for the selected model, corpus, and local run—not a general accuracy claim.

These four queries form a compact evaluation set, not a restriction of the service. The semantic endpoint accepts any non-empty natural-language query and an optional result limit, so further concepts can be evaluated directly through the API without changing the application.

The outputs also included irrelevant terms. For example, the tool query ranked `unnailed` and `knotter` ahead of `hammer`. This illustrates that vector similarity can be influenced by word associations and corpus composition. It also demonstrates why top-five recall was a more appropriate MVP measure than claiming that the first result would always be correct.

6.3 Interpretation

The experiment demonstrates the intended capability: semantic retrieval can find a target whose literal name is absent from the query. The lexical baseline remains preferable for known prefixes, while the semantic endpoint addresses descriptive discovery. The result supports completing a broader evaluation but does not establish production readiness.

7 Trustworthy AI, ethics, and governance

This POC does not make autonomous decisions and does not generate factual answers. Nevertheless, ranked retrieval can still mislead users if similarity is presented as correctness or confidence.

7.1 Misleading relevance

A high-ranked term may not satisfy the user's actual intent. The application therefore returns ranked catalog suggestions and similarity scores without presenting them as correct answers or model confidence. I manually reviewed the first five results and retained weak matches in the evaluation. Users may still over-trust the ranking when domain context is unavailable.

7.2 Ambiguous or context-poor data

Isolated dictionary words can have several meanings and provide little context to the embedding model. I documented the corpus as synthetic and unsuitable for production-quality claims. Recording irrelevant neighbours rather than removing them made this limitation visible. A production catalog would require governed descriptions and metadata to reduce the ambiguity.

7.3 Corpus coverage and evaluation bias

Retrieval cannot return a term absent from the indexed catalog. I used a deterministic sample distributed through the dictionary and verified that the expected terms were present before evaluation. Because those known terms were explicitly included, the sample may favour the prepared queries. The results demonstrate functional retrieval, not an unbiased quality benchmark.

7.4 Model and vector compatibility

Vectors generated by different models or with different dimensions cannot be compared reliably. The provider contract includes model identity and dimensions, the same provider embeds catalog terms and queries, and a reset import rebuilds the Redis index. Index inspection confirmed the expected 384 dimensions. Operational safeguards would still be needed to prevent an uncoordinated model change in production.

7.5 Credential exposure

Credentials and environment-specific configuration could be exposed through source code or terminal history. Configuration is externalized, and secrets are kept out of source, Compose, Make targets, and documentation. Shell-history hygiene remains the operator's responsibility.

7.6 Data privacy and intellectual property

A real catalog might contain personal, client, or proprietary information. This POC uses a public local dictionary and no sensitive content. A production implementation would still require data classification, access control, retention rules, and confirmation that source material may be processed by the selected model provider.

7.7 Reproducibility drift

Model and container updates can change ranking behaviour. I fixed the model name in configuration, preserved evaluation queries and measured outputs, and reran the local workflow end to end. Stronger long-term reproducibility would require pinning image digests and recording model and index versions with each run.

Human oversight is inherent in this MVP: the service returns suggestions for a person or consuming application to assess. No result triggers an external action. A production implementation would additionally require authenticated access, audit logging, monitored relevance metrics, controlled model versions, governed source data, and a review process for quality regressions.

8 Technical outcomes and lessons learned

The project delivered a working end-to-end semantic retrieval path rather than only an embedding API experiment. It demonstrated model inference, batch ingestion, binary vector handling, index lifecycle management, KNN retrieval, and comparison with a conventional baseline. The local approach also allowed development and evaluation to proceed without external credentials.

The most important implementation lesson was that model integration alone does not create useful evidence. The original catalog sample consisted of the first 1,000 alphabetically sorted dictionary lines, which concentrated almost all terms near the beginning of the alphabet. I replaced it with a deterministic distributed sample because semantic quality cannot be evaluated when expected targets are absent. This change improved the validity of the demonstration but also introduced a limitation: explicitly included evaluation terms make the current result a functional POC, not an unbiased benchmark.

The most valuable next improvements are:

1. pin container image versions rather than using `latest`;
2. replace isolated dictionary words with governed catalog records containing short descriptions and metadata;
3. create a larger evaluation set separated from corpus-selection decisions;
4. measure latency distributions and top-k retrieval metrics over repeated runs; and
5. add model and index metadata to the stored catalog so incompatible changes fail explicitly.

The project required scope control, baseline comparison, provider decoupling, binary data integration, repeatable operation, evidence-based iteration, and explicit treatment of residual limitations. These are the principal technical outcomes documented by the source code, architecture diagram, evaluation results, and demonstration workflow.

9 Supporting work products

1. **E-01 — Spring Boot source code.** Demonstrates catalog import, provider abstraction, vector encoding, Redis indexing, and lexical and semantic APIs. The source uses synthetic data and contains no credentials.
2. **E-02 — Architecture sequence diagram.** Shows the end-to-end import and semantic-query interactions among the user, API, model, and Redis. It is safe to disclose.
3. **E-03 — README and demonstration guide.** Document the architecture, repeatable operation, presentation workflow, and known limitations. They are safe to disclose.
4. **E-04 — Makefile and Compose configuration.** Provide a repeatable local Redis and Ollama environment and prepared evaluation workflow. They contain no secrets.
5. **E-05 — Evaluation results and Redis index output.** Record the measured top-five outcomes, vector-index configuration, memory usage, and zero indexing failures. The preserved queries and outputs support reproduction of the observations.
6. **E-06 — Provider integration code.** The provider-neutral interface and separate Ollama and watsonx.ai adapters show how model hosting is isolated from ingestion and retrieval. Environment-backed configuration keeps IBM Cloud credentials and project-specific values outside the repository.

10 References

The following official documentation supports the model, inference API, and vector-search details used in this report. All links were accessed on 24 September 2026.

1. IBM, [Granite Embedding models](#). Model family overview and embedding use cases.
2. IBM Granite, [Granite Embedding 30M English model card](#). Model architecture, 384-dimensional output, context length, and evaluation information.
3. Ollama, [Generate embeddings API](#). Request and response contract used by the local embedding adapter.
4. Redis, [Vector search concepts](#). Vector field types, index algorithms, distance metrics, and K-nearest-neighbour search.
5. Spring, [Spring Data Redis reference documentation](#). Redis connectivity and data-access facilities used by the application.
6. IBM, [watsonx.ai text embeddings API](#). Cloud embedding capability integrated by the watsonx.ai provider adapter.